Bring Your Own Key & Bring Your Own Subscription

An orchestration architecture for multi-provider LLM access — a start-to-deployment procedure built on free-tier services

Item	Value
Document type	Implementation procedure / reference architecture
Prepared	7 November 2026
Audience	Engineers building a SaaS or internal tool that calls LLM providers using customer-owned credentials
Reference stack	Cloudflare Workers + Pages, Supabase (Postgres + Auth), Upstash Redis, GitHub Actions — all on free tiers
Code language	TypeScript (Workers runtime), SQL (Postgres)

Verify. Free-tier quotas, provider APIs, model identifiers and terms of service change often. Every number in this document was checked against vendor pages around the preparation date where noted, and is marked 'approx.' where it was not. Re-check each vendor's current pricing and limits page before you rely on it. Code samples are illustrative starting points and have not been run against live provider accounts; test them before production use.

Contents

1. Scope, definitions and assumptions

2. Requirements and design principles

3. Reference architecture

4. Free-tier service selection

5. Phase 0 — Accounts, repository and tooling

6. Phase 1 — Data model (Supabase Postgres)

7. Phase 2 — Authentication and tenancy

8. Phase 3 — Credential vault (envelope encryption)

9. Phase 4 — BYOK onboarding and key validation

10. Phase 5 — BYOS onboarding (cloud subscriptions)

11. Phase 6 — Provider adapter layer

12. Phase 7 — Orchestrator: resolution, routing, fallback

13. Phase 8 — Rate limiting, budgets and metering

14. Phase 9 — Frontend

15. Phase 10 — Testing

16. Phase 11 — CI/CD and deployment

17. Phase 12 — Operations: observability, rotation, incidents

18. Security checklist

19. Scaling beyond free tiers

20. Go-live checklist

Appendix A — Environment variables and secrets

Appendix B — Open questions to settle for your context

1. Scope, definitions and assumptions

This procedure describes how to build and deploy an **orchestration layer** that sits between your application and one or more large-language-model (LLM) providers, where the provider credentials belong to your *customer*, not to you. The orchestrator authenticates the end user, finds the right customer credential, applies policy, calls the provider, and records usage — without ever exposing the raw secret to the browser or to logs.

1.1 Definitions used in this document

Term	Meaning here
BYOK — Bring Your Own Key	The customer pastes a provider API key (e.g. an OpenAI, Anthropic or Google AI Studio key). Your platform stores it encrypted and uses it on their behalf. Provider bills the customer.
BYOS — Bring Your Own Subscription	The customer connects an existing **cloud or enterprise subscription** they already pay for — e.g. an AWS account with Bedrock enabled, an Azure subscription with an Azure OpenAI resource, or a Google Cloud project with Vertex AI. Access is granted by a scoped, revocable mechanism (cross-account role, endpoint + key, workload identity) rather than a long-lived personal key where possible. Also covers OAuth-style 'connect your account' flows that a provider or aggregator explicitly offers to third-party apps.
Platform credential	Your own key, used only as an optional fallback or for a free trial.
Tenant	An organisation/workspace in your app. Credentials attach to a tenant or a user.
KEK / DEK	Key-encryption key (master secret) / data-encryption key (per tenant).

Verify. Consumer chat subscriptions are out of scope. Plans such as ChatGPT Plus or Claude Pro are consumer products; to my knowledge their terms do not permit reusing them as an API backend inside a third-party service, and scraping or proxying their web sessions is very likely a terms-of-service violation. I am not certain of every vendor's current wording, so read each provider's terms yourself. This document treats BYOS strictly as the sanctioned routes described above.

1.2 Assumptions (change these if they do not match your situation)

- You are building a web application (SaaS or internal) with multiple tenants.
- You want a zero-cost starting point and accept free-tier limits (cold starts, quotas, pausing) during development and early usage.
- TypeScript on Cloudflare Workers is acceptable for the backend. Section 19 lists container-based alternatives if you need Python or long-running processes.
- You are not yet subject to a specific compliance regime (SOC 2, HIPAA, PCI). If you are, free tiers are usually unsuitable for production data — see Section 19.

2. Requirements and design principles

2.1 Functional requirements

- F1. A tenant admin can add, test, rotate and delete BYOK keys per provider.
- F2. A tenant admin can connect a cloud subscription (BYOS) for Bedrock, Azure OpenAI or Vertex AI.
- F3. Application code calls one internal endpoint (`POST /v1/chat`) with a provider-neutral payload; the orchestrator chooses credential and provider.

- F4. Credential precedence is configurable: user key > tenant BYOS > tenant BYOK > platform fallback (if enabled).
- F5. Automatic fallback to another provider/credential on rate-limit or server errors, within tenant policy.
- F6. Per-tenant usage metering (tokens, requests, estimated cost) and optional budgets.
- F7. Streaming responses (server-sent events) pass through to the client.

2.2 Non-functional requirements

- N1. Secrets are encrypted at rest with authenticated encryption and never returned to clients after creation (show only a fingerprint such as the last 4 characters).
- N2. Secrets never appear in logs, error messages, analytics or crash reports.
- N3. Tenant isolation is enforced in the database (row-level security), not only in code.
- N4. Every credential use is auditable (who, when, which credential, which model).
- N5. The system degrades gracefully when a free-tier quota is exhausted.

2.3 Design principles

- **Decrypt late, forget early.** Decrypt a credential only inside the request that uses it; never cache plaintext across requests.
- **Least privilege for BYOS.** Ask the customer for a role that can only invoke models, with an external ID, instead of broad cloud credentials.
- **Adapters, not conditionals.** One adapter per provider behind a common interface keeps the router simple and testable.
- **Server-side only.** The browser never sees decrypted keys; it only ever talks to your orchestrator.

3. Reference architecture

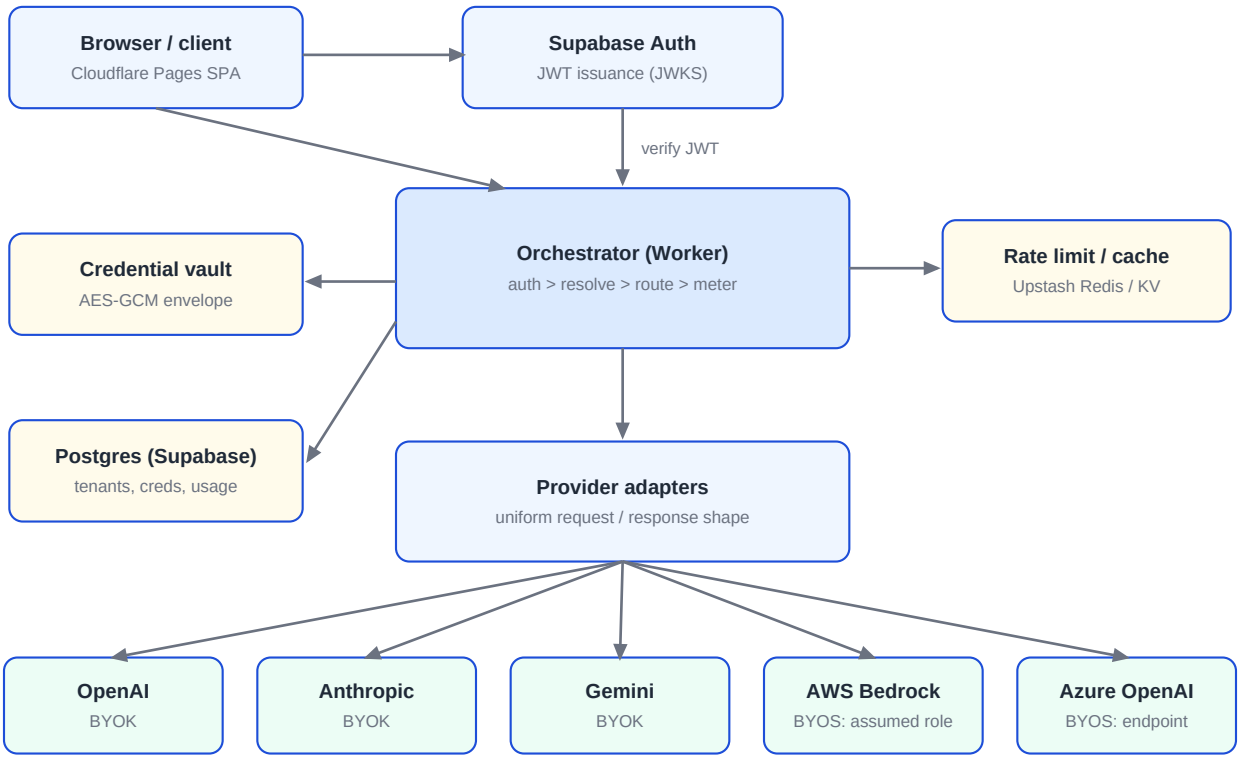
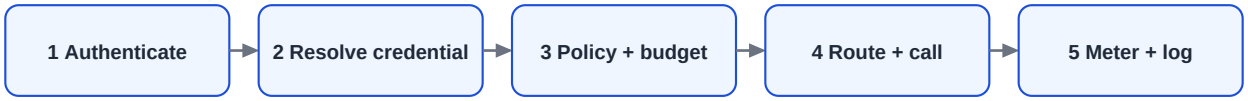


Figure 1 — Components and data flow. Amber = stateful stores; green = customer-owned provider accounts.

3.1 Components

Component	Responsibility	Free-tier implementation
Client SPA	Settings UI for keys/subscriptions; chat UI	Cloudflare Pages
Identity	Sign-up, login, JWT issuance, org membership	Supabase Auth
Orchestrator	JWT verification, credential resolution, policy, routing, fallback, streaming, metering	Cloudflare Worker
Credential vault	Envelope-encrypt / decrypt secrets; KEK held as a Worker secret	Web Crypto API inside the Worker + ciphertext in Postgres
Relational store	Tenants, members, credential metadata + ciphertext, usage, audit	Supabase Postgres
Rate limiter / counters	Per-tenant request limits, short-lived counters	Upstash Redis (or Workers KV for coarse limits)
Provider adapters	Translate neutral request to OpenAI / Anthropic / Gemini / Bedrock / Azure OpenAI / Vertex formats	Code inside the Worker
CI/CD	Tests, migrations, deploy	GitHub Actions

3.2 Request lifecycle



1. **Authenticate** — verify the Supabase JWT; load the user's tenant and role.
2. **Resolve credential** — apply precedence rules (F4) to pick a credential record that supports the requested model family.
3. **Policy + budget** — check model allow-list, rate limit and monthly budget.
4. **Route + call** — decrypt the secret, call the provider via its adapter, stream back; on 429/5xx try the next eligible credential.
5. **Meter + log** — record tokens, latency, status and credential ID (never the secret).

4. Free-tier service selection

The table summarises the free tiers this design relies on. Figures were checked on or around the preparation date against vendor documentation where marked **(checked)**; others are from my general knowledge and marked **approx.** — confirm them before you plan capacity.

Service	Role	Free-tier highlights	Caveats
Cloudflare Workers	Orchestrator	100,000 requests/day; 10 ms CPU per request (time waiting on fetch/IO does not count) **(checked)**	Limits reset daily 00:00 UTC; over-quota requests fail. Heavy JSON work can hit the CPU cap.
Workers KV	Coarse counters, config cache	100,000 reads/day, 1,000 writes/day, ~1 GB storage **(checked)**	Eventually consistent — not suitable for exact rate limits or secrets of record.
Cloudflare Pages	Frontend hosting	Static hosting with generous bandwidth (approx.; build-minute caps apply)	Verify current build limits.
Supabase	Postgres + Auth	500 MB database per project; 2 active projects **(checked)**	Free projects pause after ~1 week of inactivity and must be restored manually.
Upstash Redis	Rate limiting	Approx. 500K commands/month and 256 MB (sources conflicted)	Older docs said 10K commands/day — confirm on Upstash pricing page.
GitHub Actions	CI/CD	Free for public repos; a monthly minute allowance for private repos (approx.)	Verify current minute allowance for your account type.
Render (alternative)	Container backend	750 free instance-hours/month per workspace; spins down after 15 min idle, ~1 min wake **(checked)**	Cold starts hurt interactive latency.
Google Cloud Run (alt.)	Container backend	Monthly free request/CPU allowance (approx. 2M requests/month)	Requires a billing account; verify current free allowance.
Oracle Cloud Always Free (alt.)	Self-managed VM	Arm VM capacity in the Always Free tier (approx. up to 4 OCPU / 24 GB)	Capacity availability varies by region; you run the OS and patches.

Note. Vercel's Hobby plan is free but, as far as I know, restricted to non-commercial use; check its current terms before using it for a commercial product. Fly.io ended its general free allowance for new organisations in 2024 to my knowledge — verify before choosing it.

4.1 Why Workers for the orchestrator

- The orchestrator is I/O-bound: most wall-clock time is spent waiting on the LLM provider, which does not count toward the Workers CPU limit.
- No cold-start sleep like free container platforms; global edge placement.
- Built-in Web Crypto for AES-GCM, and first-class secrets via `wrangler secret put`.
- Streaming responses are supported through standard `ReadableStream`.

5. Phase 0 — Accounts, repository and tooling

5.1 Create accounts

1 Cloudflare (Workers + Pages), Supabase, Upstash, GitHub.

2 Enable MFA on every account; these accounts guard your customers' keys.

3 Create separate **staging** and **production** Supabase projects (the free tier allows two active projects).

5.2 Repository layout

```
byok-orchestrator/
  apps/
    web/                 # SPA (Vite + React or similar) -> Cloudflare Pages
    orchestrator/        # Cloudflare Worker (TypeScript)
      src/
        index.ts         # router / entry
        auth.ts          # JWT verification
        vault.ts         # envelope encryption
        resolve.ts       # credential resolution
        adapters/        # openai.ts, anthropic.ts, gemini.ts, bedrock.ts, azure.ts
        meter.ts         # usage + audit
        ratelimit.ts
      wrangler.toml
  supabase/
    migrations/          # SQL migrations (schema + RLS)
  .github/workflows/     # CI + deploy
```

5.3 Bootstrap the Worker

```
npm create cloudflare@latest apps/orchestrator   # choose "Hello World" Worker, TypeScript
cd apps/orchestrator
npm install jose @upstash/redis @upstash/ratelimit aws4fetch
npx wrangler login
```

`jose` verifies JWTs, `@upstash/ratelimit` implements sliding-window limits, and `aws4fetch` signs AWS SigV4 requests for Bedrock/STS in a fetch-based runtime. Check each package's README for its current API before use.

5.4 Minimal wrangler.toml

```
name = "byok-orchestrator"
main = "src/index.ts"
compatibility_date = "2026-10-01"     # set to the date you start; see Cloudflare docs

[vars]
SUPABASE_URL = "https://<project-ref>.supabase.co"
ALLOWED_ORIGIN = "https://app.example.com"

# Secrets (set with `npx wrangler secret put NAME`, never committed):
#   KEK_V1                    base64 32-byte master key
#   SUPABASE_SERVICE_ROLE_KEY server-only DB key
#   UPSTASH_REDIS_REST_URL / UPSTASH_REDIS_REST_TOKEN
#   PLATFORM_FALLBACK_KEY     optional
```

6. Phase 1 — Data model (Supabase Postgres)

Create a migration under `supabase/migrations/` and apply it with the Supabase CLI (`supabase db push`) or the SQL editor.

```
create table tenants (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  policy      jsonb not null default '{"precedence":["user","byos","byok","platform"],
                                       "allow_platform_fallback": false}',
  monthly_budget_usd numeric,
  created_at  timestamptz not null default now()
);

create table memberships (
  tenant_id uuid references tenants(id) on delete cascade,
  user_id   uuid references auth.users(id) on delete cascade,
  role      text not null check (role in ('owner','admin','member')),
  primary key (tenant_id, user_id)
);

create table credentials (
  id           uuid primary key default gen_random_uuid(),
  tenant_id    uuid not null references tenants(id) on delete cascade,
  owner_user   uuid references auth.users(id),          -- null = tenant-wide
  kind         text not null check (kind in ('byok','byos')),
  provider     text not null,   -- openai | anthropic | gemini | bedrock | azure_openai | vertex
  label        text,
  config       jsonb not null default '{}',  -- non-secret: region, endpoint, role ARN, models
  ciphertext   bytea not null,  -- AES-GCM(secret JSON)
  iv           bytea not null,
  wrapped_dek  bytea not null,  -- DEK encrypted under KEK
  kek_version  int  not null default 1,
  fingerprint  text not null,   -- e.g. 'sk-...a1b2'
  status       text not null default 'active' check (status in ('active','invalid','revoked')),
  last_validated_at timestamptz,
  created_at   timestamptz not null default now()
);

create table usage_events (
  id            bigserial primary key,
  tenant_id     uuid not null references tenants(id) on delete cascade,
  user_id       uuid,
  credential_id uuid references credentials(id) on delete set null,
  provider      text, model text,
  input_tokens  int, output_tokens int,
  est_cost_usd  numeric, latency_ms int, status int,
  created_at    timestamptz not null default now()
);

create table audit_log (
```

```
  id bigserial primary key, tenant_id uuid, actor uuid,
  action text not null, target uuid, meta jsonb, at timestamptz default now()
);
```

6.1 Row-level security

The Worker uses the service-role key (which bypasses RLS), but RLS still protects any direct client access through the Supabase JS client. Critically, the client must never be able to read `ciphertext` or `wrapped_dek`.

```
alter table credentials  enable row level security;
alter table usage_events enable row level security;
alter table memberships  enable row level security;

-- Members may see their own memberships
create policy own_memberships on memberships for select
  using (user_id = auth.uid());

-- Expose credential METADATA only, through a view without secret columns
create view credential_public with (security_invoker = true) as
  select id, tenant_id, kind, provider, label, config, fingerprint, status,
         last_validated_at, created_at
  from credentials;

create policy members_read_creds on credentials for select
  using (tenant_id in (select tenant_id from memberships where user_id = auth.uid()));

-- No insert/update/delete policies for clients: all writes go through the Worker.
revoke select (ciphertext, iv, wrapped_dek) on credentials from anon, authenticated;
```

Verify. Column-level REVOKE combined with RLS and views has subtle behaviour in PostgreSQL; test with the `authenticated` role that a client query for `ciphertext` fails. The simplest safe option is to deny all client access to `credentials` and serve metadata only via the Worker.

7. Phase 2 — Authentication and tenancy

The SPA signs users in with Supabase Auth and sends the access token as `Authorization: Bearer <jwt>` to the Worker. The Worker verifies it locally against the project's JWKS.

```
// src/auth.ts
import { createRemoteJWKSet, jwtVerify } from "jose";

let jwks: ReturnType<typeof createRemoteJWKSet> | undefined;

export async function authenticate(req: Request, env: Env) {
  const token = req.headers.get("Authorization")?.replace(/^Bearer\s+/i, "");
  if (!token) throw new HttpError(401, "missing token");
  jwks ??= createRemoteJWKSet(new URL(`${env.SUPABASE_URL}/auth/v1/.well-known/jwks.json`));
  const { payload } = await jwtVerify(token, jwks, {
    issuer: `${env.SUPABASE_URL}/auth/v1`,
    audience: "authenticated",
  });
  const tenantId = req.headers.get("X-Tenant-Id");
  const role = await db.membershipRole(env, tenantId, payload.sub!);   // query memberships
  if (!role) throw new HttpError(403, "not a member of tenant");
  return { userId: payload.sub!, tenantId: tenantId!, role };
}
```

Verify. Supabase has used both symmetric (HS256 shared secret) and asymmetric (JWKS) JWT signing; which one your project uses depends on its settings and creation date. Confirm the JWKS URL and issuer in your project's Auth settings, or verify with the legacy JWT secret if your project still uses HS256.

8. Phase 3 — Credential vault (envelope encryption)

Each credential secret is encrypted with a random 256-bit **DEK** using AES-256-GCM. The DEK itself is encrypted ("wrapped") with the **KEK**, which exists only as a Worker secret. A database leak alone therefore yields no usable keys. Storing `kek_version` lets you rotate the KEK without downtime.

```
// src/vault.ts  — uses only the standard Web Crypto API available in Workers
const b64 = (s: string) => Uint8Array.from(atob(s), c => c.charCodeAt(0));

async function kek(env: Env, version: number): Promise<CryptoKey> {
  const raw = b64((env as any)[`KEK_V${version}`]);
  return crypto.subtle.importKey("raw", raw, "AES-GCM", false, ["encrypt", "decrypt"]);
}

export async function seal(env: Env, secret: object, aad: string) {
  const dekRaw = crypto.getRandomValues(new Uint8Array(32));
  const dek = await crypto.subtle.importKey("raw", dekRaw, "AES-GCM", false, ["encrypt"]);
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const ad = new TextEncoder().encode(aad);            // bind to tenant+credential id
  const ct = await crypto.subtle.encrypt({ name: "AES-GCM", iv, additionalData: ad }, dek,
              new TextEncoder().encode(JSON.stringify(secret)));
  const wiv = crypto.getRandomValues(new Uint8Array(12));
  const wrapped = await crypto.subtle.encrypt({ name: "AES-GCM", iv: wiv },
              await kek(env, env.KEK_CURRENT_VERSION), dekRaw);
  return { ciphertext: new Uint8Array(ct), iv,
           wrapped_dek: concat(wiv, new Uint8Array(wrapped)),
           kek_version: env.KEK_CURRENT_VERSION };
}

export async function open(env: Env, rec: SealedRecord, aad: string) {
  const wiv = rec.wrapped_dek.slice(0, 12), wct = rec.wrapped_dek.slice(12);
  const dekRaw = await crypto.subtle.decrypt({ name: "AES-GCM", iv: wiv },
              await kek(env, rec.kek_version), wct);
  const dek = await crypto.subtle.importKey("raw", dekRaw, "AES-GCM", false, ["decrypt"]);
  const pt = await crypto.subtle.decrypt(
      { name: "AES-GCM", iv: rec.iv, additionalData: new TextEncoder().encode(aad) },
      dek, rec.ciphertext);
  return JSON.parse(new TextDecoder().decode(pt));
}
```

8.1 Generate and install the KEK

```
openssl rand -base64 32 | npx wrangler secret put KEK_V1
# also add KEK_CURRENT_VERSION = "1" (as a var or secret)
```

- Store an offline backup of the KEK in a password manager or hardware-backed vault. **If the KEK is lost, every stored credential is unrecoverable.**
- Use a different KEK for staging and production.
- Use additional authenticated data (AAD) = `tenant_id:credential_id` so a ciphertext copied into another tenant's row fails to decrypt.
- Upgrade path: replace the Worker-secret KEK with a managed KMS (AWS KMS, GCP KMS, HashiCorp Vault) when you leave free tiers.

9. Phase 4 — BYOK onboarding and key validation

9.1 Flow

1. Admin opens Settings > AI Providers > Add key; selects provider; pastes key over HTTPS.
2. Worker validates the key with a cheap, read-only provider call (list models).

3 On success: seal, insert row, write audit event, return only metadata + fingerprint.

4 The plaintext key is dropped from memory at the end of the request; the UI never shows it again.

9.2 Validation endpoints

Provider	Validation call	Auth header
OpenAI	GET https://api.openai.com/v1/models	Authorization: Bearer <key>
Anthropic	GET https://api.anthropic.com/v1/models	x-api-key: <key>; anthropic-version: 2023-06-01
Google Gemini (AI Studio)	GET https://generativelanguage.googleapis.com/v1beta/models	x-goog-api-key: <key>
OpenRouter (aggregator)	Check its docs for a key-info endpoint	Authorization: Bearer

Verify. Endpoint paths and header names above are correct to the best of my knowledge but providers do change them — confirm against each provider's current API reference.

```
// src/routes/credentials.ts (excerpt)
export async function addByok(req: Request, env: Env, ctx: AuthCtx) {
  requireRole(ctx, ["owner", "admin"]);
  const { provider, apiKey, label } = await req.json<AddKeyBody>();
  if (!VALIDATORS[provider]) throw new HttpError(400, "unsupported provider");
  const ok = await VALIDATORS[provider](apiKey);        // list-models call
  if (!ok) throw new HttpError(422, "key rejected by provider");
  const id = crypto.randomUUID();
  const sealed = await seal(env, { apiKey }, `${ctx.tenantId}:${id}`);
  await db.insertCredential(env, { id, tenant_id: ctx.tenantId, kind: "byok", provider,
    label, ...sealed, fingerprint: `${apiKey.slice(0, 3)}...${apiKey.slice(-4)}`,
    last_validated_at: new Date().toISOString() });
  await audit(env, ctx, "credential.create", id, { provider });
  return json({ id, provider, label, status: "active" }, 201);
}
```

10. Phase 5 — BYOS onboarding (cloud subscriptions)

BYOS lets enterprises keep spend, data-residency and quota inside their own cloud agreement. Prefer mechanisms that are **scoped, revocable, and short-lived**.

10.1 AWS Bedrock — cross-account IAM role (recommended pattern)

1 Your platform owns one AWS IAM user (or, better, a role reachable via OIDC) whose only permission is `sts:AssumeRole`. Store its credentials as Worker secrets.

2 Generate a unique **external ID** per tenant and show it in the onboarding screen.

3 The customer creates a role in *their* account that trusts your principal with that external ID and grants only Bedrock invoke permissions on the models they choose.

4 The customer pastes the role ARN and region; you call STS AssumeRole to test, then store ARN + region + external ID in `config` (the external ID is not a secret but treat it carefully).

5 Per request: AssumeRole (cache the temporary credentials for less than their expiry), then SigV4-sign the Bedrock runtime call.

```
// Customer-side trust policy (they create this in their AWS account)
{
  "Version": "2012-10-17",
  "Statement": [{
```

```
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::<YOUR_ACCOUNT_ID>:user/byok-orchestrator" },
    "Action": "sts:AssumeRole",
    "Condition": { "StringEquals": { "sts:ExternalId": "<TENANT_EXTERNAL_ID>" } }
  }]
}
// Customer-side permissions policy (scope Resource to specific model ARNs if desired)
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
    "Resource": "*" }] }

// src/adapters/bedrock.ts (excerpt) — aws4fetch signs requests with SigV4
import { AwsClient } from "aws4fetch";

async function assume(env: Env, cfg: BedrockCfg) {
  const sts = new AwsClient({ accessKeyId: env.AWS_PLATFORM_KEY_ID,
    secretAccessKey: env.AWS_PLATFORM_SECRET, service: "sts", region: cfg.region });
  const url = `https://sts.${cfg.region}.amazonaws.com/?Action=AssumeRole&Version=2011-06-15` +
    `&RoleArn=${encodeURIComponent(cfg.roleArn)}&RoleSessionName=byok-${cfg.tenantId}` +
    `&ExternalId=${encodeURIComponent(cfg.externalId)}&DurationSeconds=900`;
  const res = await sts.fetch(url);
  return parseAssumeRoleXml(await res.text());   // AccessKeyId, SecretAccessKey, SessionToken
}
// Then call https://bedrock-runtime.<region>.amazonaws.com/model/<modelId>/converse
// with a new AwsClient({ ...tempCreds, sessionToken, service: "bedrock" }).
```

Verify. The Bedrock Converse endpoint path, STS query parameters, and aws4fetch constructor options are given from memory; confirm them in the AWS Bedrock Runtime API reference, the STS AssumeRole reference, and the aws4fetch README.

10.2 Azure OpenAI — customer resource endpoint

- Customer supplies the resource endpoint, deployment name(s), API version and either an API key or (preferred, more work) a Microsoft Entra ID app registration they grant the 'Cognitive Services OpenAI User' role (role name to be verified).
- Store the endpoint/deployments in `config`; seal the key or client secret in the vault.
- Validate by calling a cheap endpoint on the customer's resource.

10.3 Google Vertex AI — service account or workload identity federation

- Simplest: customer creates a service account with Vertex AI user permissions and uploads a JSON key; you seal the whole JSON. Many organisations block key creation by policy.
- Better: Workload Identity Federation, where Google trusts an identity token you issue, avoiding long-lived keys. This needs an OIDC issuer you control — more setup; verify Google's current guidance.
- In a Worker you must mint an OAuth access token from the service-account key yourself (sign a JWT with RS256 via Web Crypto, exchange it at Google's token endpoint).

10.4 Aggregator OAuth ("connect your account")

Some aggregators (OpenRouter is one I believe offers this — verify) provide an OAuth/PKCE flow that returns a user-scoped key to your app. Treat the returned key exactly like a BYOK key: seal it, fingerprint it, and allow the user to revoke it.

11. Phase 6 — Provider adapter layer

Define one neutral request/response shape and implement it once per provider. The router never contains provider-specific code.

```
// src/adapters/types.ts
export interface ChatRequest {
  model: string;                 // logical name, e.g. "fast" | "smart" or a concrete id
  messages: { role: "system" | "user" | "assistant"; content: string }[];
  maxTokens?: number; temperature?: number; stream?: boolean;
}
export interface ChatResult {
  stream?: ReadableStream<Uint8Array>; text?: string;
  usage?: { inputTokens: number; outputTokens: number };
  status: number; retryable: boolean;
}
export interface Adapter {
  provider: string;
  supports(model: string): boolean;
  call(req: ChatRequest, secret: any, cfg: any, signal: AbortSignal): Promise<ChatResult>;
}

// src/adapters/anthropic.ts (non-streaming excerpt)
export const anthropic: Adapter = {
  provider: "anthropic",
  supports: m => m.startsWith("claude-"),
  async call(req, secret, _cfg, signal) {
    const system = req.messages.filter(m => m.role === "system").map(m => m.content).join("\n");
    const res = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST", signal,
      headers: { "x-api-key": secret.apiKey, "anthropic-version": "2023-06-01",
                 "content-type": "application/json" },
      body: JSON.stringify({ model: req.model, max_tokens: req.maxTokens ?? 1024,
        system: system || undefined,
        messages: req.messages.filter(m => m.role !== "system") }),
    });
    const retryable = res.status === 429 || res.status >= 500;
    if (!res.ok) return { status: res.status, retryable };
    const j: any = await res.json();
    return { status: 200, retryable: false,
      text: j.content?.map((b: any) => b.text ?? "").join(""),
      usage: { inputTokens: j.usage?.input_tokens, outputTokens: j.usage?.output_tokens } };
  },
};

// src/adapters/openai.ts (non-streaming excerpt)
export const openai: Adapter = {
  provider: "openai",
  supports: m => /^(gpt-|o\d)/.test(m),
  async call(req, secret, _cfg, signal) {
    const res = await fetch("https://api.openai.com/v1/chat/completions", {
      method: "POST", signal,
      headers: { Authorization: `Bearer ${secret.apiKey}`, "content-type": "application/json" },
      body: JSON.stringify({ model: req.model, messages: req.messages,
                             max_tokens: req.maxTokens, temperature: req.temperature }),
    });
    const retryable = res.status === 429 || res.status >= 500;
    if (!res.ok) return { status: res.status, retryable };
    const j: any = await res.json();
    return { status: 200, retryable: false, text: j.choices?.[0]?.message?.content,
      usage: { inputTokens: j.usage?.prompt_tokens, outputTokens: j.usage?.completion_tokens } };
  },
};
```

Verify. Request/response field names reflect these providers' public APIs as I know them; OpenAI in particular also offers a newer Responses API and some models reject `max_tokens` in favour of another parameter. Model identifiers change frequently — keep them in configuration, not code, and confirm current IDs in each provider's model list.

11.1 Streaming

For `stream: true`, request the provider's SSE stream and pipe it through a `TransformStream` that (a) normalises events into one format for your client, and (b) captures the final usage event for metering. Return `new Response(readable, { headers: { 'content-type': 'text/event-stream' } })`. Use `ctx.waitUntil()` to write usage after the stream completes.

11.2 Logical model aliases

Let tenants map aliases (`fast`, `smart`, `cheap`) to concrete model IDs per provider in `tenants.policy`. This makes fallback across providers possible without the application knowing provider-specific names.

12. Phase 7 — Orchestrator: resolution, routing, fallback

```
// src/resolve.ts
export async function candidates(env: Env, ctx: AuthCtx, req: ChatRequest) {
  const tenant = await db.tenant(env, ctx.tenantId);
  const creds = await db.activeCredentials(env, ctx.tenantId);    // includes sealed fields
  const order: string[] = tenant.policy.precedence;              // ["user","byos","byok",...]
  const ranked = [];
  for (const tier of order) {
    for (const c of creds) {
      const tierOf = c.owner_user ? "user" : c.kind;              // user | byos | byok
      if (tierOf !== tier) continue;
      if (tier === "user" && c.owner_user !== ctx.userId) continue;
      const model = resolveAlias(tenant.policy, req.model, c.provider);
      if (model && ADAPTERS[c.provider].supports(model)) ranked.push({ cred: c, model });
    }
  }
  if (tenant.policy.allow_platform_fallback && env.PLATFORM_FALLBACK_KEY)
    ranked.push(platformCandidate(env, tenant, req));
  return ranked;
}

// src/index.ts — core handler (simplified)
export default {
  async fetch(req: Request, env: Env, ctx: ExecutionContext) {
    if (req.method === "OPTIONS") return cors(env, new Response(null, { status: 204 }));
    try {
      const auth = await authenticate(req, env);
      const url = new URL(req.url);
      if (url.pathname === "/v1/chat" && req.method === "POST") {
        const body = await req.json<ChatRequest>();
        await enforceLimits(env, auth);                          // rate limit + budget
        const list = await candidates(env, auth, body);
        if (!list.length) throw new HttpError(409, "no credential configured for this model");
        for (const { cred, model } of list.slice(0, 3)) {     // cap fallback attempts
          const secret = await open(env, cred, `${cred.tenant_id}:${cred.id}`);
          const t0 = Date.now();
          const r = await ADAPTERS[cred.provider].call({ ...body, model }, secret, cred.config,
                      AbortSignal.timeout(60_000));
          ctx.waitUntil(meter(env, auth, cred, model, r, Date.now() - t0));
          if (r.status === 401 || r.status === 403) ctx.waitUntil(markInvalid(env, cred.id));
          if (r.status === 200) return cors(env, respond(r));
          if (!r.retryable) return cors(env, json({ error: "provider_error",
                                                    status: r.status }, 502));
        }
        return cors(env, json({ error: "all_candidates_failed" }, 503));
      }
```

```
      return routeAdmin(req, env, auth);                        // credential CRUD etc.
    } catch (e) { return cors(env, errorResponse(e)); }       // never echo secrets
  },
};
```

12.1 Routing rules worth implementing

- **Fail closed on auth errors:** a 401/403 from a provider marks the credential `invalid` and notifies the tenant admin — do not silently fall back to the platform key, or you will pay for their usage.
- **Retry only idempotent failures:** 429, 500, 502, 503, 504, and network timeouts.
- **Respect Retry-After** when present, but do not block a Worker for long; prefer moving to the next candidate.
- **Data-residency pinning:** a tenant policy flag can restrict candidates to BYOS credentials in specific regions.
- **No fallback mid-stream:** once bytes have been streamed to the client, a failure must surface as an error event rather than a silent provider switch.

13. Phase 8 — Rate limiting, budgets and metering

```
// src/ratelimit.ts
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis/cloudflare";

export async function enforceLimits(env: Env, auth: AuthCtx) {
  const rl = new Ratelimit({
    redis: Redis.fromEnv(env),                          // reads UPSTASH_REDIS_REST_URL/TOKEN
    limiter: Ratelimit.slidingWindow(60, "1 m"),        // 60 req / minute / tenant
    prefix: "rl",
  });
  const { success } = await rl.limit(`tenant:${auth.tenantId}`);
  if (!success) throw new HttpError(429, "tenant rate limit");
  if (await overBudget(env, auth.tenantId)) throw new HttpError(402, "monthly budget reached");
}
```

- **Cost estimate** = tokens x a price table you maintain in config (prices change; store the table with an effective date). For BYOK/BYOS the provider bills the customer, so the estimate is informational unless you resell.
- **Budget check**: keep a running month-to-date total in Redis (`INCRBYFLOAT`) updated in `waitUntil`; reconcile nightly from `usage_events`.
- **Quota awareness**: at roughly 100K Worker requests/day and ~500K Upstash commands/month (approx.), one rate-limit check per request would consume the Upstash allowance before the Worker allowance at sustained load — budget commands accordingly or batch checks.

13.1 Data retention

The Supabase free database is 500 MB. A usage row is small (very roughly a few hundred bytes including indexes — measure yours), so millions of rows will approach the cap. Schedule a job (GitHub Actions cron or a Workers Cron Trigger) to roll up rows older than N days into daily aggregates and delete the raw rows. Do **not** store prompts or completions by default.

14. Phase 9 — Frontend

- Settings > AI Providers page lists credentials from the Worker's metadata endpoint (provider, label, fingerprint, status, last validated).
- Add-key form: password-type input, `autocomplete="off"`, submit directly to the Worker; clear the field from state after submit.
- BYOS wizard per provider: shows your AWS account ID and the tenant's external ID, copyable policy JSON, and a 'Test connection' button.
- Precedence and fallback toggles mapped to `tenants.policy`.
- Usage dashboard reading aggregated usage from the Worker.
- Content-Security-Policy restricting `connect-src` to your Worker and Supabase domains; no third-party scripts on the settings page.

```
# Deploy SPA to Cloudflare Pages
cd apps/web && npm run build
npx wrangler pages deploy dist --project-name byok-web
```

15. Phase 10 — Testing

Layer	What to test	How
Vault	Round-trip; wrong AAD fails; wrong KEK version fails; tampered ciphertext fails	Unit tests with Vitest (Cloudflare provides a Workers Vitest integration — verify setup)
Adapters	Request mapping; error classification (401 vs 429 vs 500); usage parsing	Mock fetch with recorded fixtures; no live keys in CI
Resolver	Precedence ordering; user-scoped keys not visible to other users; alias mapping	Table-driven unit tests
RLS	Client cannot read ciphertext or other tenants' rows	SQL tests against a local Supabase (supabase start) using the authenticated role
Logging	No secret in logs	Test that greps captured logs for a canary key string
End-to-end	Add key, chat, rotate, revoke	Playwright against staging with test keys you own and spend caps set at the provider
Load	Behaviour at quota exhaustion	Small k6/autocannon run against staging; confirm graceful 429/503 rather than 500

16. Phase 11 — CI/CD and deployment

16.1 Secrets setup (once per environment)

```
npx wrangler secret put KEK_V1                     --env production
npx wrangler secret put SUPABASE_SERVICE_ROLE_KEY  --env production
npx wrangler secret put UPSTASH_REDIS_REST_URL     --env production
npx wrangler secret put UPSTASH_REDIS_REST_TOKEN   --env production
npx wrangler secret put AWS_PLATFORM_KEY_ID        --env production   # only if using Bedrock BYOS
npx wrangler secret put AWS_PLATFORM_SECRET        --env production
```

In GitHub: add repository secrets `CLOUDFLARE_API_TOKEN` (scoped to Workers/Pages edit for one account), `CLOUDFLARE_ACCOUNT_ID`, and `SUPABASE_ACCESS_TOKEN` / DB password for migrations. Use GitHub Environments with required reviewers for production.

16.2 Workflow

```
# .github/workflows/deploy.yml
name: deploy
on:
  push: { branches: [main] }
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci && npm test --workspaces
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: supabase/setup-cli@v1
      - run: supabase link --project-ref ${{ secrets.SUPABASE_PROJECT_REF }}
             && supabase db push
        env:
          SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }}
          SUPABASE_DB_PASSWORD: ${{ secrets.SUPABASE_DB_PASSWORD }}
      - uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
          accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
          workingDirectory: apps/orchestrator
          command: deploy --env production
```

Verify. Action versions and inputs (`cloudflare/wrangler-action`, `supabase/setup-cli`) should be checked against their READMEs and pinned to a commit SHA for supply-chain safety.

16.3 Deployment procedure

1. Merge to `main` after tests pass on the pull request.
2. Pipeline applies database migrations first (they must be backward-compatible with the currently running Worker).
3. Pipeline deploys the Worker; Cloudflare switches traffic to the new version.
4. Deploy the SPA to Pages (separate job or Pages Git integration).
5. Smoke test: health endpoint, sign in, add a test key, run one chat, check a usage row exists.
6. Rollback: `npx wrangler rollback` (verify the command for your Wrangler version) or redeploy the previous commit; database migrations should be forward-only with expand/contract steps.

16.4 Custom domain and TLS

Attach `api.example.com` to the Worker and `app.example.com` to Pages in the Cloudflare dashboard; TLS certificates are issued automatically when the domain's DNS is on Cloudflare. Set `ALLOWED_ORIGIN` to the app origin for CORS.

17. Phase 12 — Operations

17.1 Observability

- Enable Workers Logs / observability in `wrangler.toml` and use `npx wrangler tail` for live debugging.
- Structured log fields: request ID, tenant ID, credential ID, provider, model, status, latency. Never the key, prompt or completion.
- A redaction helper that strips anything matching key patterns (`sk-`, `AKIA`, long base64 strings) before logging errors.
- Optional free-tier error tracking (e.g. Sentry's free developer plan — verify limits) with request bodies disabled.

17.2 Keep-alive for free tiers

Supabase free projects pause after about a week without activity. A scheduled GitHub Action or Workers Cron Trigger that performs a trivial authenticated query every few days prevents this. Check whether Supabase's terms permit this for your use; upgrading is the robust fix.

17.3 Rotation

What	Procedure
Customer key (BYOK)	Customer adds new key, tests it, sets it active, deletes old. Offer periodic re-validation (daily cron calling list-models) and alert on failure.
KEK	Add KEK_V2; set KEK_CURRENT_VERSION=2; run a batch job that unwraps each DEK with V1 and re-wraps with V2 (ciphertext itself is unchanged); when no rows use V1, delete KEK_V1.
Platform AWS principal	Create new access key, update secret, deploy, deactivate old key. Customers' trust policies are unaffected because the principal ARN is unchanged.
Supabase service-role key	Rotate in Supabase, update Worker secret, redeploy.

17.4 Incident runbook (suspected credential exposure)

1. Contain: disable the affected tenant's credentials (status = revoked) via an admin endpoint.
2. Notify the customer promptly so they revoke the key at the provider — only they can.
3. If the KEK may be exposed: rotate it immediately (17.3) and ask all tenants to rotate keys.
4. Review audit_log and usage_events for anomalous use; preserve logs.
5. Meet any contractual or legal notification duties applicable to you (seek legal advice; I cannot determine these for your jurisdiction).

18. Security checklist

- [ ] KEK only in Worker secrets + offline backup; distinct per environment.
- [ ] AES-GCM with random 96-bit IVs, never reused; AAD binds ciphertext to tenant and credential.
- [ ] Plaintext secrets never returned by any API, never logged, never stored in KV or Redis.

- [ ] Client has no read access to ciphertext columns (tested).
- [ ] Only owners/admins can create, rotate or delete credentials; all actions audited.
- [ ] BYOS uses scoped roles with external IDs; no customer root/admin credentials accepted.
- [ ] Provider 401/403 marks credential invalid; no automatic fallback to platform billing.
- [ ] CORS restricted to your app origin; CSP on the frontend; HTTPS only.
- [ ] Request size limits and per-tenant rate limits to prevent abuse of customer keys.
- [ ] Dependencies pinned; GitHub Actions pinned to SHAs; Dependabot or similar enabled.
- [ ] Privacy policy and terms disclose that you store and use customer credentials.

19. Scaling beyond free tiers

Pressure point	Symptom	Upgrade path
Worker request quota	Errors after ~100K requests/day	Workers Paid plan
Worker CPU limit	Error 1102 on large payload transforms	Paid plan (higher CPU limits) or move heavy transforms off the hot path
Supabase pausing / 500 MB	Project paused; insert failures	Supabase Pro, or another managed Postgres
KEK in env secret	Compliance asks for HSM-backed keys	Cloud KMS / Vault with envelope encryption; Worker calls KMS to unwrap DEKs
Need Python / long jobs	Agentic workflows exceeding request time	Container platform (Cloud Run, Render paid, Fly.io) with a queue; keep the vault logic identical
Compliance (SOC 2 etc.)	Customer security reviews	Paid tiers with compliance attestations; formal key-management policy; pen test

20. Go-live checklist

- [ ] Staging and production fully separated (projects, KEKs, API tokens).
- [ ] All Section 18 items verified.
- [ ] Migrations applied; RLS tests passing in CI.
- [ ] Smoke test passed on production with an owned test key; test key then removed.
- [ ] Monitoring and alerting for 5xx rate, quota exhaustion and invalid-credential spikes.
- [ ] Keep-alive or paid plan in place for any component that pauses.
- [ ] Rollback rehearsed once on staging.
- [ ] Runbook (17.4) shared with whoever is on call.
- [ ] Free-tier limits in Section 4 re-checked against vendor pages on launch day.

Appendix A — Environment variables and secrets

Name	Type	Where	Purpose
SUPABASE_URL	var	Worker	Project base URL
SUPABASE_SERVICE_ROLE_KEY	secret	Worker	Server-side DB access (bypasses RLS)
KEK_V1 .. KEK_Vn	secret	Worker	Master key(s) for wrapping DEKs
KEK_CURRENT_VERSION	var	Worker	Which KEK wraps new DEKs
UPSTASH_REDIS_REST_URL / _TOKEN	secret	Worker	Rate limiting / counters
AWS_PLATFORM_KEY_ID / _SECRET	secret	Worker	Principal allowed to AssumeRole (BYOS)
PLATFORM_FALLBACK_KEY	secret	Worker	Optional platform-paid fallback
ALLOWED_ORIGIN	var	Worker	CORS origin of the SPA
VITE_SUPABASE_URL / _ANON_KEY	public	SPA build	Client auth (anon key is public by design)
CLOUDFLARE_API_TOKEN / ACCOUNT_ID	secret	GitHub	Deploy
SUPABASE_ACCESS_TOKEN / DB_PASSWORD / PROJECT_REF	secret	GitHub	Migrations

Appendix B — Open questions to settle for your context

These materially change the design; I have assumed defaults above but you should decide them explicitly:

- Which providers must be supported at launch, and which BYOS clouds your customers actually use?
- Is there a platform-paid fallback (and therefore billing/reselling), or strictly customer-paid?
- Do you need to store prompts/completions (e.g. for history), which raises data-protection obligations?
- Which compliance regimes and data-residency constraints apply to your customers?
- Do you need per-user keys, per-tenant keys, or both?
- Expected request volume — does it fit inside the free quotas in Section 4?